

Failure Modes in Chained Tool-Calling Agents Operating Without Human Checkpoints

Abstract

Abstract Large language model (LLM)-based agents increasingly combine reasoning, external information retrieval, memory, and tool invocation into iterative workflows. Unlike conventional single-turn language-model applications, these systems can transform the output of one operation into the input, argument, or justification for a subsequent operation. Such chained tool calling substantially expands agent capability, but it also creates failure modes that are difficult to detect when human checkpoints are removed. This paper examines the principal reliability and security failures that emerge in autonomous chains of tool calls. Drawing on recent literature on tool-augmented language models, autonomous-agent benchmarks, prompt injection, agent security, and verifiable tool use, the paper develops a lifecycle-oriented taxonomy covering planning errors, argument and schema errors, state and context drift, error propagation, feedback-loop failures, hallucinated tool results, excessive agency, indirect prompt injection, privilege escalation, data exfiltration, memory poisoning, and failures of verification. A central observation is that chain length changes the risk profile: even when individual tool calls have relatively high reliability, correlated errors and unverified intermediate states can produce substantially lower end-to-end reliability. Existing approaches—including ReAct-style reasoning, self-reflection, instruction hierarchies, task-alignment defenses, runtime policy enforcement, and benchmark-based evaluation—address portions of this problem but generally do not provide strong guarantees for long-horizon autonomous execution. The paper argues that future research should move from model-centric safeguards toward system-level assurance based on provenance tracking, capability isolation, state invariants, causal attribution of actions, risk-adaptive checkpoints, transactional execution, and formal verification of tool-call sequences. The resulting research agenda treats human oversight not as the only safety mechanism, but as one component of a broader architecture in which autonomous actions remain bounded, auditable, reversible, and aligned with the user’s objective.

Keywords: LLM agents; tool calling; autonomous agents; chained execution; agent safety; prompt injection; tool misuse; long-horizon reasoning; human oversight; AI security; verification; provenance

1. Introduction

Large language models have progressed from systems that primarily generate or transform text into systems capable of interacting with external environments. Tool-augmented models can invoke search engines, calculators, databases, code interpreters, browsers, APIs, file systems, and other software services. Toolformer demonstrated that language models can learn when to invoke APIs, what arguments to provide, and how to incorporate returned information into subsequent generation [1]. ReAct subsequently established a widely influential paradigm in which reasoning and action are interleaved, allowing a model to gather observations from an environment and update its behavior accordingly [2]. These developments form an important foundation for contemporary LLM agents. The distinction between an isolated tool call and a chain of tool calls is particularly important. In a chained agent, an initial model decision can change the state of the environment, and the resulting observation can influence the next decision. Formally, an agent can be represented as a sequence $a_t = \pi_{\theta}(G, S_t, O_{\leq t}, M_t)$, where G is the user’s goal, S_t is the current environment state, $O_{\leq t}$ represents observations accumulated up to time t , M_t denotes persistent or episodic memory, and a_t is the next action or tool call. The environment then produces O_{t+1} , $S_{t+1} = E(S_t, a_t)$. The next action is therefore conditioned not only on the original instruction but also on the entire history of model-generated decisions and externally supplied observations. This recursive structure creates opportunities for errors to propagate. The problem becomes more consequential when human checkpoints are absent. A human operator can often detect a mistaken interpretation, an unexpected side effect, or a suspicious request before it becomes irreversible. Without such intervention, an agent may continue executing a flawed plan. A small error at step t can therefore become a prerequisite for a larger error at $t+1$, and a later action may appear locally reasonable even though the chain as a whole has already departed from the original objective. Empirical benchmarks demonstrate that long-horizon agent performance remains substantially below human performance. WebArena reported that its best GPT-4-based agent achieved 14.41% end-to-end success compared with 78.24% for humans on its realistic web tasks [3]. AgentBench similarly identified long-term reasoning, decision-making, and instruction following as major obstacles to usable LLM agents [4]. OSWorld later showed substantial deficiencies in

open-ended computer-use environments, where even strong multimodal agents struggled with GUI grounding and operational knowledge [5]. More recent long-horizon evaluation continues to indicate that agents lose track of constraints, miss information appearing during execution, and fail to verify intermediate states [6]. At the same time, tool access introduces a security dimension that is distinct from ordinary model hallucination. External content can contain instructions that manipulate subsequent actions. Greshake et al. demonstrated that indirect prompt injection can remotely influence LLM-integrated applications by embedding adversarial instructions in retrieved data [7]. InjecAgent subsequently showed that tool-integrated agents remain vulnerable to indirect prompt injection, while AgentDojo established a dynamic environment for systematically evaluating attacks and defenses [8,9]. Thus, a chained tool-calling agent can fail either because its reasoning is incorrect or because an adversary deliberately manipulates the information on which subsequent tool decisions depend. This paper investigates these failures as a systems problem. Rather than treating hallucination, prompt injection, planning failure, and unsafe tool use as independent phenomena, it considers how they interact over a sequence of autonomous actions. The paper has four principal objectives: (1) to establish a taxonomy of failure modes in chained tool-calling agents; (2) to distinguish reliability failures from security failures while showing where they interact; (3) to critically examine current mitigation and evaluation approaches; and (4) to identify research directions for safer autonomous execution without requiring a human approval step before every action. The central thesis is that the primary safety challenge is not merely whether an individual tool call is correct. It is whether the entire sequence of actions remains aligned with the original objective while preserving state integrity, authorization boundaries, data provenance, and recoverability.

2. Background and Related Work

2.1 Tool-Augmented Language Models

Tool use extends the computational capabilities of language models beyond their learned parameters. Toolformer demonstrated a framework in which models learn to determine whether an external API should be called and how its result should be incorporated into generation [1]. API-Bank subsequently introduced a benchmark containing 73 runnable APIs and 753 annotated API calls, highlighting planning, API retrieval, and API execution as distinct capabilities [10]. The basic tool-calling loop can be described as: receive a goal; determine whether a tool is needed; select a tool; construct arguments; execute the tool; observe its result; update the working state; and decide whether to continue. The observation and state-update stages are critical for chained execution. A tool result is not merely an output; it becomes new model context. If the result is erroneous, stale, malicious, ambiguous, or incorrectly interpreted, the subsequent decision may also be wrong.

2.2 Reasoning and Acting

ReAct proposed interleaving reasoning and actions so that an agent can use external observations to update its plan [2]. This approach is attractive for long-horizon tasks because it avoids committing to a complete plan before the environment is observed. However, interleaving reasoning and action also creates a feedback dependency. The model's previous actions determine the observations available to the next reasoning step. Consequently, a mistaken action can alter the information landscape in which later decisions are made. The resulting failure is therefore not necessarily a sequence of independent mistakes; it can be a self-reinforcing trajectory. Reflexion explored another strategy by allowing language agents to generate verbal feedback and retain reflective information for subsequent attempts [11]. Such methods can improve performance by introducing a form of self-evaluation, but self-reflection itself remains model-generated and cannot automatically guarantee that the underlying evaluation is correct.

2.3 Autonomous-Agent Benchmarks

Several benchmarks have shifted evaluation from isolated question answering toward interactive task completion. AgentBench evaluates agents across multiple environments and reports weaknesses in long-term reasoning, decision-making, and instruction following [4]. WebArena provides realistic websites and long-horizon tasks across domains such as e-commerce, social forums, software development, and content management [3]. OSWorld extends this direction to real operating-system environments and evaluates tasks involving multiple applications [5]. These benchmarks demonstrate an important methodological point: single-step accuracy is insufficient to characterize autonomous-agent reliability. A model can correctly perform many individual actions but still fail to complete a task because one incorrect decision occurs late in the chain. SWE-bench provides a related perspective for software-engineering agents. It evaluates models on 2,294 real GitHub issues requiring modifications across real repositories and found that the benchmark was substantially more difficult than conventional code-generation tasks

[12]. This illustrates how stateful environments, dependencies across files, and iterative execution introduce difficulties that are absent from isolated generation.

2.4 Security of Tool-Using Agents

The security literature has identified several risks that become more severe when agents can act autonomously. Greshake et al. demonstrated indirect prompt injection, in which malicious instructions are embedded in retrieved content rather than supplied directly by the user [7]. InjecAgent systematically evaluated indirect prompt injection in tool-integrated agents using 1,054 test cases covering 17 user tools and 62 attacker tools [8]. AgentDojo extended the evaluation setting by providing 97 realistic tasks and 629 security test cases involving external tools and untrusted data [9]. Agent Security Bench (ASB) further broadened the scope across 10 scenarios, more than 400 tools, and multiple attack and defense mechanisms, identifying vulnerabilities across system prompts, user prompts, tool usage, and memory retrieval [13].

3. Failure Modes in Chained Tool-Calling Agents

3.1 Planning and Goal-Decomposition Failure

The first failure class occurs before or during tool selection. A high-level instruction may be underspecified, ambiguous, or decomposed incorrectly. An agent might transform a user's objective into a sequence that is locally coherent but globally incorrect. For example, an instruction to “find the cheapest suitable flight and prepare the booking” can be incorrectly reduced to search, select cheapest, book. Here “suitable” may imply baggage allowance, timing, cancellation conditions, or a preferred airport. Selecting the cheapest result before resolving these constraints creates a downstream state in which later actions operate on the wrong candidate. This is a form of goal drift: the operational objective becomes narrower or different from the user's actual objective. AgentBench's findings concerning long-term reasoning and decision-making reinforce the importance of this problem [4].

3.2 Tool-Selection Failure

Even when the goal is correctly understood, an agent can select the wrong tool. Multiple tools may provide overlapping functionality but have different permissions, side effects, precision, or reliability. A semantic model may recognize `read_customer_record`, `update_customer_record`, `send_email`, and `delete_customer_record` as related to customer management but fail to distinguish their consequences under a particular instruction. Tool-selection errors are especially dangerous because the model's natural-language interface can hide the security significance of a capability.

3.3 Argument-Construction and Schema Failure

A selected tool can still be invoked incorrectly. Common errors include wrong argument types, missing required fields, incorrect identifiers, invalid date or time formats, wrong units, incorrect account or recipient, stale identifiers, and semantically valid but contextually inappropriate values. The distinction between syntactic validity and semantic validity is crucial. A tool call can satisfy its JSON schema and execute successfully while still being wrong.

3.4 State-Tracking Failure

Long chains require the agent to maintain an accurate representation of the environment. The model must know what has already happened, what remains to be done, and which observations are current. State errors include forgetting a previous action, confusing two entities, assuming an action succeeded without confirmation, using stale information, failing to notice an environmental change, and incorrectly reconstructing the state after an error. OSWorld 2.0 provides relevant evidence that long-horizon tasks reveal failures involving hidden state, missed information, constraint tracking, and skipped verification [6].

3.5 Error Propagation and Compounding

The defining characteristic of chained tool calling is that errors can propagate. Let p_i denote the probability that step i is correct, assuming independence for illustration. Then the probability that all n steps are correct is $P(\text{all correct}) = \prod_i p_i$. If each step has reliability $p=0.98$, then for 50 sequential steps, 0.98^{50} is approximately 0.364. This does not imply that real agent failures obey an independent model; errors are often correlated. The calculation instead illustrates why high per-action accuracy does not automatically yield high end-to-end reliability.

More importantly, real chains can exhibit conditional amplification: $P(E_{t+1}|E_t) > P(E_{t+1})$, where E_t represents an earlier error. Once the state becomes corrupted, later actions can become more likely to fail.

3.6 Feedback-Loop Failure

A particularly dangerous pattern occurs when the agent uses the consequences of its own incorrect action as evidence that the action was correct. For example, the agent incorrectly modifies a configuration, queries the configuration, receives the modified value, and interprets that value as confirmation of the intended state. The observation is factually correct but semantically misleading. The tool reports what is, not whether what is present matches the user's intended state. This produces a distinction between execution verification and goal verification.

3.7 Hallucinated or Misinterpreted Tool Results

Tool use does not eliminate hallucination. Instead, hallucination can move from factual generation into interpretation. An agent may correctly receive a tool result but incorrectly summarize, filter, or reason about it. A related failure occurs when a tool returns an ambiguous or partial result and the model fills the missing information using assumptions.

3.8 Instruction and Context Drift

A long execution history can contain system policies, developer constraints, user instructions, tool documentation, tool results, retrieved documents, web pages, emails, memory, and previous model-generated plans. The model must distinguish between information and instructions. The Instruction Hierarchy approach explicitly addresses this issue by training models to prioritize instructions according to their authority [14]. However, hierarchy alone does not solve every chained-agent problem because malicious content may exploit legitimate instructions or produce tool outputs that appear operationally relevant.

3.9 Indirect Prompt Injection

Indirect prompt injection is among the most important failure modes in autonomous tool chains. An attacker does not necessarily need to interact with the model directly. Instead, malicious instructions can be placed in a document, web page, email, database record, or API response that the agent later reads. The attack can be represented as untrusted data -> agent observation -> instruction interpretation -> tool call. Greshake et al. demonstrated that retrieved prompts can manipulate application functionality and API usage [7]. InjecAgent confirmed that such attacks can cause direct harm or private-data exfiltration in tool-integrated agents [8].

3.10 Excessive Agency and Privilege Escalation

Another failure occurs when an agent possesses more capabilities than are necessary for its task. If a model has access to email, databases, payment systems, file storage, and shell commands, a single compromised reasoning trajectory can cross multiple security boundaries. This resembles the classical confused deputy problem. Therefore, capability exposure should be minimized according to the principle that available capabilities should approximate required capabilities.

3.11 Data Exfiltration

A chained agent can unintentionally transform a read capability into an exfiltration capability. For example: read private file -> summarize content -> send external request. Each individual action may be permitted. The composition is the security violation. This illustrates why evaluating tools individually is insufficient. Security properties may emerge only from the relationship between consecutive calls.

3.12 Memory Poisoning

Agents increasingly use persistent memory to maintain user preferences, prior interactions, plans, or environmental knowledge. Memory can improve continuity but introduces another attack surface. An attacker may insert a malicious or incorrect memory item that affects future tool calls: poisoned memory -> future retrieval -> planning bias -> unsafe action. ASB explicitly evaluates memory-related attacks and identifies memory retrieval as one stage at which agent security can fail [13].

4. A Lifecycle Taxonomy of Chained-Agent Failures

The principal lifecycle stages are goal interpretation, planning, tool selection, argument construction, observation, state update, memory, verification, authorization, chain composition, and recovery. Representative failures include goal drift, incorrect decomposition, wrong capability, invalid parameters, malicious or ambiguous output, state drift, memory poisoning, false confirmation, excessive privileges, cross-tool interaction, and repeated failed strategies. A complete evaluation should therefore consider goal alignment, planning correctness, tool selection, argument correctness, state integrity, observation integrity, and verification quality rather than a single accuracy number.

5. Current Approaches and Developments

5.1 ReAct-Style Execution

ReAct remains influential because it explicitly combines reasoning with environmental interaction [2]. Its advantage is adaptability, but the same feedback loop can propagate incorrect observations and decisions. ReAct improves flexibility without automatically providing assurance.

5.2 Self-Reflection and Iterative Correction

Reflexion introduces verbal feedback into the agent loop [11]. It can reduce repeated failures, but self-generated reflection is not independent verification. If the model incorrectly diagnoses the failure, reflective memory can reinforce the wrong explanation.

5.3 Instruction Hierarchies

Instruction hierarchy distinguishes trusted system-level instructions from lower-priority user or third-party instructions [14]. This is useful against instruction conflicts, but hierarchy cannot by itself determine whether a permitted action is appropriate.

5.4 Task Alignment Verification

Task Shield proposes verifying whether each instruction and tool call contributes to the user's objective [16]. On AgentDojo, the authors report an attack success rate of 2.07% while retaining 69.79% task utility on GPT-4o [16]. This represents a shift toward checking the relationship between an action and the user's task.

5.5 Runtime Policy Enforcement

Runtime enforcement places a security boundary between the model and the tool. Recent work on runtime policy enforcement for MCP-based agents combines tool-call interception with information-flow labels and tamper-evident logging [17]. Such architectures move critical controls outside the probabilistic model.

5.6 Causal Attribution of Tool Calls

AttriGuard represents a recent direction: instead of asking whether a tool call looks malicious, it asks whether the call is causally supported by the user's intent and whether it survives removal or attenuation of untrusted observations [18]. This is relevant to indirect prompt injection because an injected instruction may influence an action without being explicitly identifiable as malicious.

5.7 Formal and Verifiable Safety

Recent research has begun moving toward formal guarantees for tool-use safety. Doshi et al. propose deriving safety requirements using System-Theoretic Process Analysis and enforcing constraints over capabilities, confidentiality, and tool sequences [15]. This represents a shift from model safety toward system safety.

6. Research Challenges and Limitations

Major challenges include long-horizon evaluation, correlated failures, hidden state, verification versus execution, security-utility trade-offs, and the lack of standardized failure taxonomies. Future benchmarks should measure when the first error occurred, whether it was recoverable, whether the agent detected it, whether subsequent actions amplified it, whether the final outcome was achieved accidentally, and whether a security invariant was violated before recovery.

7. Research Gaps and Future Directions

7.1 Risk-Adaptive Human Checkpoints

The absence of human checkpoints should not necessarily mean the absence of all oversight. A better architecture is risk-adaptive autonomy, where low-risk actions execute automatically while high-risk transitions trigger verification.

7.2 Transactional Tool Chains

Tool calls should increasingly support transaction-like semantics: plan, simulate, validate, commit, verify, and rollback where possible. Irreversible operations should require stronger authorization and verification.

7.3 Provenance-Aware Context

Every observation should carry provenance metadata describing source, trust level, timestamp, authorization scope, whether it contains instructions, and whether it has been externally validated. This preserves the distinction between trusted instruction, untrusted data, tool result, and agent-generated hypothesis.

7.4 State Invariants

Agents should operate under explicit invariants, such as never sending data outside approved domains, never modifying records outside the user's authorization scope, and never executing payment without verified beneficiary information. Invariants provide a stronger basis for safety than asking a language model to be careful.

7.5 Causal Action Attribution

Future systems should explain each high-impact tool call in terms of its causal relationship to the user's objective. If an action depends primarily on an untrusted observation rather than the user's objective, the system should increase its risk score or block execution.

7.6 Compositional Security Verification

Security analysis should move from individual tools to tool compositions. Two individually safe functions can form an unsafe composition if their combined information flow violates confidentiality.

7.7 Failure-Aware Memory

Memory systems should distinguish facts from decisions, observations from instructions, and trusted user preferences from untrusted external content. Each memory item should ideally contain content, source, timestamp, trust, scope, and expiry.

7.8 Better Benchmarks for Chain-Level Reliability

Future benchmarks should report task success rate, first-error position, error recovery rate, unsafe-action rate, constraint-violation rate, and verification coverage. Risk-weighted chain failure rate is another useful metric.

8. Conclusion

Chained tool-calling agents represent an important transition from language generation toward autonomous computation and interaction. Their capabilities derive from repeatedly reasoning, acting, observing, and acting again. Yet this iterative structure is also their principal source of risk. The literature demonstrates that current agents remain unreliable on long-horizon tasks, even when their underlying language models perform strongly on conventional benchmarks [3–6,12]. Tool integration creates security vulnerabilities including indirect prompt injection, tool misuse, data exfiltration, excessive agency, and memory poisoning [7–9,13]. These risks become more severe when no human checkpoint interrupts the execution chain. The most important conceptual conclusion is that agent safety cannot be reduced to the correctness of individual tool calls. The relevant unit of analysis is the complete action trajectory. A chain can fail through incorrect planning, stale state, malicious observations, inappropriate tool composition, authorization violations, or insufficient verification. Current approaches provide valuable partial solutions. ReAct improves interaction with environments; Reflexion enables self-correction; instruction hierarchies strengthen resistance to conflicting instructions; task-alignment defenses examine whether actions serve the user's objective; runtime policy enforcement places controls outside the model; and emerging formal-verification and

causal-attribution approaches seek stronger guarantees [2,11,14–18]. None, however, fully resolves the challenge of safe long-horizon autonomous execution. Ultimately, the goal should not be to build agents that merely make fewer mistakes. It should be to build agents whose mistakes are bounded, detectable, reversible, attributable, and unable to cross predefined safety and authorization boundaries. Achieving that objective requires treating autonomous tool calling as a systems-engineering and verification problem as much as a language-modeling problem. References [1] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language Models Can Teach Themselves to Use Tools,” *Advances in Neural Information Processing Systems (NeurIPS)*,

2023. arXiv:2302.04761. DOI: 10.48550/arXiv.2302.04761.

[2] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “ReAct: Synergizing Reasoning and Acting in Language Models,” *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629. DOI: 10.48550/arXiv.2210.03629. [3] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A Realistic Web Environment for Building Autonomous Agents,” *International Conference on Learning Representations (ICLR)*,

2024. arXiv:2307.13854. DOI: 10.48550/arXiv.2307.13854.

[4] X. Liu et al., “AgentBench: Evaluating LLMs as Agents,” *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2308.03688. DOI: 10.48550/arXiv.2308.03688. [5] T. Xie et al., “OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2404.07972. [6] M. Yuan et al., “OSWorld2.0: Benchmarking Computer Use Agents on Long-Horizon Real-World Tasks,” arXiv, 2026. arXiv:2606.29537. [7] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz, and M. Fritz, “Not What You’ve Signed Up For: Compromising Real-World LLM-Integrated Applications with Indirect Prompt Injection,” *Proceedings of the 2023 ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90. DOI: 10.1145/3605764.3623985. arXiv:2302.12173. [8] Q. Zhan, Z. Liang, Z. Ying, and D. Kang, “InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents,” *Findings of the Association for Computational Linguistics: ACL 2024*, 2024, pp. 10471–10506. DOI: 10.18653/v1/2024.findings-acl.624. arXiv:2403.02691. [9] E. Debenedetti, J. Zhang, M. Balunovic, L. Beurer-Kellner, M. Fischer, and F. Tramèr, “AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents,” *Advances in Neural Information Processing Systems (NeurIPS 37)*, 2024, pp. 82895–82920. DOI: 10.52202/079017-2636. [10] M. Li, Y. Zhao, B. Yu, F. Song, H. Li, H. Yu, Z. Li, F. Huang, and Y. Li, “API-Bank: A Comprehensive Benchmark for Tool-Augmented LLMs,” arXiv, 2023. arXiv:2304.08244. DOI: 10.48550/arXiv.2304.08244. [11] N. Shinn, F. Cassano, E. Berman, A. Gopinath, K. Narasimhan, and S. Yao, “Reflexion: Language Agents with Verbal Reinforcement Learning,” *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366. DOI: 10.48550/arXiv.2303.11366. [12] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?” *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.06770. DOI: 10.48550/arXiv.2310.06770. [13] H. Zhang et al., “Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents,” *International Conference on Learning Representations (ICLR)*, 2025. arXiv:2410.02644. DOI: 10.48550/arXiv.2410.02644. [14] E. Wallace, K. Xiao, R. Leike, L. Weng, J. Heidecke, and A. Beutel, “The Instruction Hierarchy: Training LLMs to Prioritize Privileged Instructions,” arXiv, 2024. arXiv:2404.13208. DOI: 10.48550/arXiv.2404.13208. [15] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, “Towards Verifiably Safe Tool Use for LLM Agents,” *Proceedings of the IEEE/ACM International Conference on Software Engineering (ICSE)*, 2026. arXiv:2601.08012. DOI: 10.1145/3786582.3786839. [16] F. Jia, T. Wu, X. Qin, and A. Squicciarini, “The Task Shield: Enforcing Task Alignment to Defend Against Indirect Prompt Injection in LLM Agents,” *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2025, pp. 29680–29697. DOI: 10.18653/v1/2025.acl-long.1435. arXiv:2412.16682. [17] “Runtime Policy Enforcement for MCP-Based LLM Agents,” *Electronics*, vol. 15, 2026, article 2829. DOI: 10.3390/electronics15132829. [18] Y. He, H. Zhu, Y. Li, S. Shao, H. Yao, Z. Liu, and Z. Qin, “AttriGuard: Defeating Indirect Prompt Injection in LLM Agents via Causal Attribution of Tool Invocations,” *35th USENIX Security Symposium (USENIX Security ’26)*, 2026.